dimensionality

March 6, 2026

1 Dimensiereductie

Auteurs: Brian van der Bijl (brian.vanderbijl@hu.nl), Tijmen Muller

- Studentnummer: 1876061
- Naam: Duain Crasto
- Datum: 06/03/2026

1.1 Deel I: Principal Component Analysis (PCA)

Het *Principal Component Analysis* (PCA) algoritme kan gebruikt worden om het aantal dimensies van een dataset te reduceren tot de belangrijkste componenten. Als de originele dataset n dimensies heeft, dan kunnen we met onderstaande stappen dit terugbrengen tot een (zelfgekozen) aantal van n' dimensies.

1. Centreer de data.
2. Bereken de covariantie van alle features onderling.
3. Bereken de Eigenvectors en Eigenvalues van de covariantiematrix.
4. Kies de n' Eigenvectors om de dimensiereductie mee uit te voeren.
5. Vermenigvuldig de n' Eigenvectors met de originele data om de reductie toe te passen.

1.1.1 Context

Gegeven is een databestand met embeddings van 200 tekstfragmenten. Elke embedding bestaat in 15 dimensies, en is gelabeled met een categorie. We gaan dimensionaliteitsreductie toepassen om de data te kunnen plotten.

De categorie geeft aan in welk genre de tekstfragmenten thuishoren. Daarnaast is onderscheid gemaakt tussen het perspectief waarin het fragment geschreven is: ik (1st person) of hij/haar/hen (3rd person): - Fantasy (1st person) - Fantasy (3rd person) - Science Fiction (1st person) - Science Fiction (3rd person) - Romance (1st person) - Romance (3rd person) - Crime (1st person) - Crime (3rd person)

[2]:
```
import numpy as np
import pickle
import matplotlib.pyplot as plt

# Show floats on 3 digits, suppress scientific notation
np.set_printoptions(precision=3, suppress=True)
```

[3]:
```
with open('data.pkl', 'rb') as file:
    data = pickle.load(file)

data.sample(5)
```

[3]:
```
shape: (5, 16)


e_1        e_2        e_3        e_4        …  e_13       e_14       e_15
label
---        ---        ---        ---           ---        ---        ---
---
f64        f64        f64        f64           f64        f64        f64
str


0.865469   0.962353   0.620742   1.060326   …  0.649668   1.308703   0.697569
Crime (3rd

person)
0.750608   0.481706   0.515727   0.851599   …  0.941314   1.312962   0.933636
Romance (3rd

person)
1.73984    1.034494   1.351793   1.004043   …  0.658452   0.573585   0.507984
Romance (1st

person)
1.277668   0.188238   1.251281   0.907016   …  0.553606   0.966405   0.903978
Science Fiction

(1st person)
1.225105   1.216584   1.423586   1.248953   …  1.0466     1.127903   0.689821
Fantasy (1st

person)
```

1.1.2 Voorbereidende opdracht

Gegeven een dataset met m datapunten met elk n features en een gewenste reductie tot n' dimensies. Bepaal voor elk van de vijf stappen van het algoritme wat de dimensies (oftewel `shape`) is van de volgende tussenresultaten:

0. De matrix met de originele dataset.
1. De matrix met de gemiddelden per feature om de data mee te centreren.
2. De covariantiematrix.

3. De matrix met de Eigenvectors en de matrix met de Eigenvalues.
4. De matrix met de *geselecteerde* Eigenvectors.
5. De matrix met de *gereduceerde* data.

Antwoord:

0. (m,n)
1. (1,n)
2. (n,n)
3. Eigenvectors: (n,n), Eigenvalues: (n,)
4. (n, n')
5. (m, n')

1.1.3 Opdracht 1. Implementatie

Schrijf een eigen implementatie van het PCA-algoritme `compute_pca(X, n_components)` volgens eerdergenoemde stappen van het algoritme. Maak slim gebruik van fucties van `numpy` waar mogelijk, maar zorg wel dat je begrijpt wat je in elke stap doet. De laatste stap is al gegeven in de functiedefinitie hieronder.

Hint: Laat bij stap 3. zien (bijvoorbeeld met een `print()` statement) dat de meest informatieve Eigenvalue al meer dan 50% van de informatie bevat van onze dataset.

Input

- `X: numpy.array` - numpy matrix met dimensies (m, n); elke rij is een datapunt in n dimensies
- `n_components: int` - het gewenste aantal dimensies n'

Output `X_reduced: numpy.array` - een (m, n') numpy matrix met de gereduceerde data.

[4]:
```
def compute_pca(X, n_components):
    """
    Parameters
    ----------
    X : numpy.ndarray
        Input data matrix of shape (m, n), where m is the number of samples and␣
↪n is the number of features.
    n_components : int
        The number of principal components (dimensions) to keep.

    Returns
    -------
    X_reduced : numpy.ndarray
        The data projected onto the top n_components principal components.
    """
    arr = np.array(X)

    # TODO: PCA algoritme implementeren

    #stap 1
```

```
    gemiddelde = np.mean(arr, axis=0)
    arr_centered = arr - gemiddelde

    #stap 2
    cov_matrix = np.cov(arr_centered, rowvar=False)

    #stap 3
    eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
    # print(np.max(eigenvalues)/np.sum(eigenvalues))

    #stap 4
    sorted_indices = np.argsort(eigenvalues)[::-1]
    best_indices = sorted_indices[:n_components]
    n_eigenvectors = eigenvectors[:, best_indices]

    print(f"n_components:{n_components} : {np.sum(eigenvalues[sorted_indices[:
↪n_components]]) / np.sum(eigenvalues)})")
    # Stap 5.
    return arr_centered @ n_eigenvectors
```

Test-scenario Onderstaande code zou de volgende output moeten opleveren (het minteken kan wisselen):

```
[[ 0.43437323 -0.49820384]
 [ 0.42077249  0.50351448]
 [-0.85514571 -0.00531064]]
```

[5]:
```
np.random.seed(1)
X = np.random.rand(3, 10)
X_reduced = compute_pca(X, n_components=2)
print(X_reduced)
```

```
n_components:2 : (0.9999999999999999+0j))
[[ 0.434+0.j -0.498+0.j]
 [ 0.421+0.j  0.504+0.j]
 [-0.855+0.j -0.005+0.j]]
```

1.1.4 Opdracht 2. Visualisatie met dimensiereductie

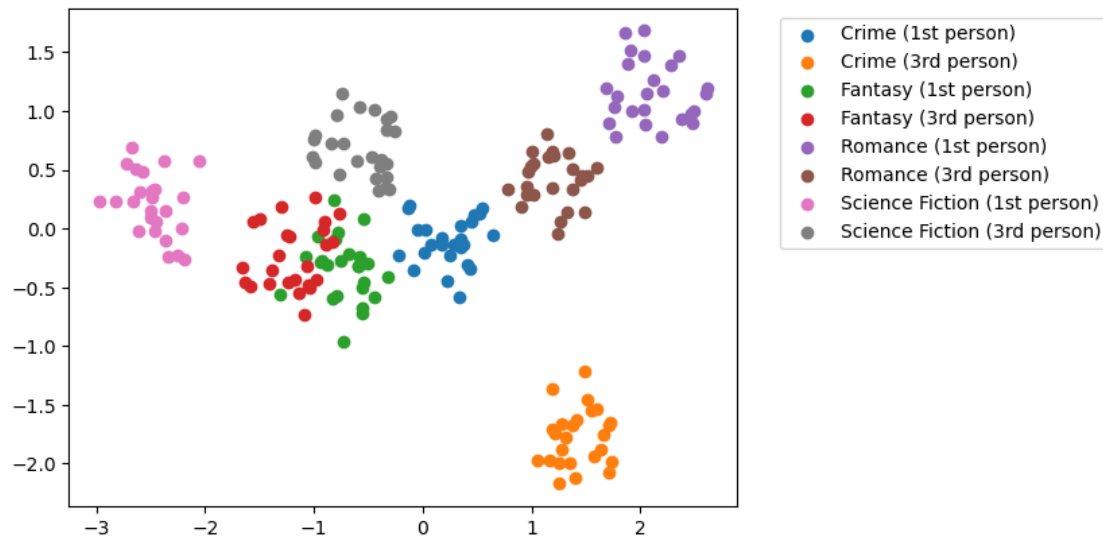
Maak op basis van de aangeleverde `data` een numpy array van de datapunten, en gebruik je PCA-implementatie om een 2D- en 3D-weergave van de data te maken. Maak van elke weergave een plot, waarbij iedere categorie een eigen kleur krijgt.

[6]:
```
# TODO: Schrijf hier je code.
filtered = [col for col in data.columns if 'e_' in col]
X = data.select(filtered).to_numpy()
labels = np.array(data['label'])
X_reduced = compute_pca(X, n_components=2)
```

```
for label in np.unique(labels):
    mask = labels == label
    plt.scatter(X_reduced[mask, 0], X_reduced[mask, 1], label=label)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
```

```
n_components:2 : 0.6866656218830466)
```

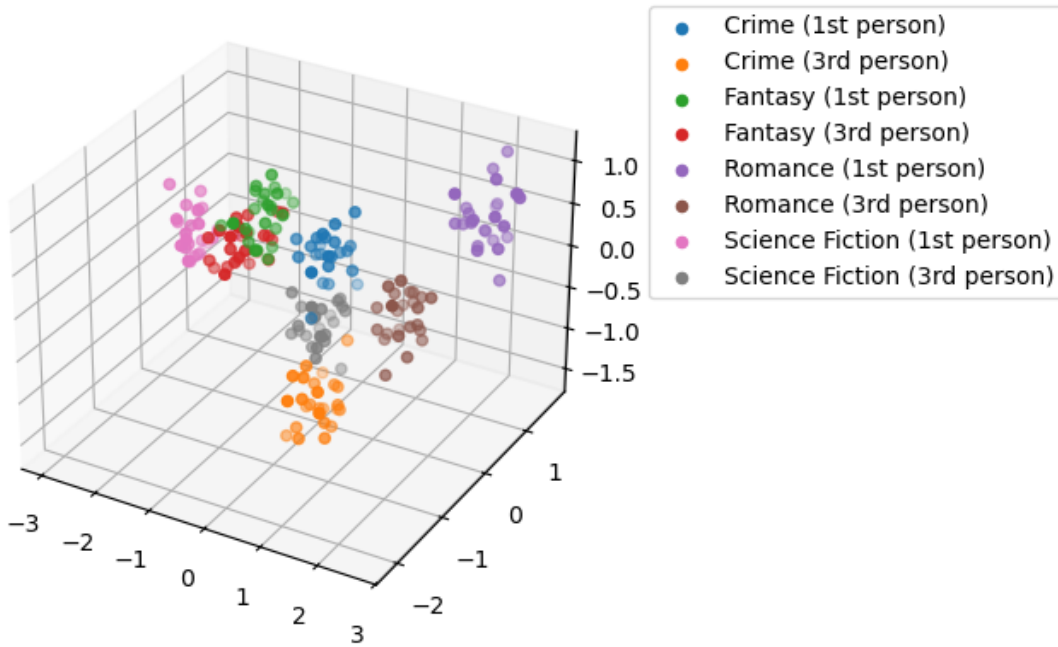
[6]: <matplotlib.legend.Legend at 0x1b774b86c30>



[7]:
```
# TODO: Schrijf hier je code.
filtered = [col for col in data.columns if 'e_' in col]
X = data.select(filtered).to_numpy()
labels = np.array(data['label'])
X_reduced = compute_pca(X, n_components=3)
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
for label in np.unique(labels):
    mask = labels == label
    ax.scatter(X_reduced[mask, 0], X_reduced[mask, 1], X_reduced[mask, 2],␣
 ↪label=label)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
```

```
n_components:3 : 0.7721763602652282)
```

[7]: <matplotlib.legend.Legend at 0x1b774e48080>



1.1.5 Opdracht 3. Analyse

Analyseer de resultaten: 1. Welke categorieën zijn op basis van de PCA-reductie te onderscheiden, en welke niet? 2. Geef aan hoeveel procent van de informatie bewaard is gebleven in 2D en 3D respectievelijk. 3. Hoeveel dimensies zijn nodig om 90% van de informatie te bewaren? 4. En voor 95%?

[8]:
```
print(f"{compute_pca(X, n_components=2)}, {compute_pca(X, n_components=3)}")
```

```
n_components:2 : 0.6866656218830466)
n_components:3 : 0.7721763602652282)
[[-0.547  0.087]
 [-0.555 -0.675]
 [-0.587 -0.319]
 [-0.546 -0.457]
 [-0.551 -0.719]
 [-0.595 -0.322]
 [-1.305 -0.557]
 [-0.675 -0.221]
 [-0.76  -0.269]
 [-0.834 -0.592]
 [-0.814  0.238]
 [-0.552 -0.504]
 [-0.968 -0.068]
 [-0.507 -0.301]
 [-0.882 -0.307]
```

```
[-0.733 -0.957]
[-0.926 -0.269]
[-1.077 -0.243]
[-0.934 -0.283]
[-0.776 -0.037]
[-0.44  -0.58 ]
[-0.584 -0.243]
[-0.32  -0.407]
[-0.803 -0.09 ]
[-0.794 -0.572]
[-1.633 -0.46 ]
[-1.303  0.187]
[-1.058 -0.317]
[-0.984  0.269]
[-1.32  -0.223]
[-0.908  0.055]
[-1.557  0.062]
[-1.174 -0.435]
[-1.227 -0.065]
[-1.093 -0.735]
[-1.255 -0.057]
[-1.492  0.086]
[-1.143 -0.553]
[-0.831 -0.11 ]
[-1.056 -0.483]
[-1.416 -0.466]
[-1.586 -0.495]
[-0.893 -0.133]
[-0.973 -0.439]
[-0.913 -0.014]
[-0.771  0.128]
[-1.387 -0.349]
[-1.653 -0.326]
[-1.234 -0.452]
[-1.033 -0.503]
[-2.457 -0.018]
[-2.501  0.146]
[-2.825  0.23 ]
[-2.2    0.268]
[-2.673  0.687]
[-2.728  0.551]
[-2.188 -0.267]
[-2.485  0.265]
[-2.22  -0.   ]
[-2.054  0.572]
[-2.368  0.153]
[-2.505  0.096]
[-2.631  0.506]
```

```
[-2.492  0.32 ]
[-2.254 -0.228]
[-2.617 -0.023]
[-2.453  0.063]
[-2.345 -0.238]
[-2.37  -0.106]
[-2.581  0.48 ]
[-2.466  0.335]
[-2.656  0.228]
[-2.597  0.312]
[-2.379  0.58 ]
[-2.971  0.232]
[-0.404  0.322]
[-0.984  0.791]
[-0.442  1.017]
[-0.429  0.428]
[-0.983  0.564]
[-0.464  0.611]
[-0.789  0.97 ]
[-0.262  0.829]
[-0.308  0.33 ]
[-0.337  0.552]
[-0.339  0.437]
[-0.729  0.723]
[-0.338  0.836]
[-0.77   0.465]
[-0.297  0.955]
[-1.01   0.609]
[-0.58   1.035]
[-0.838  0.72 ]
[-0.343  0.343]
[-0.334  0.936]
[-0.738  1.154]
[-1.002  0.759]
[-0.6    0.571]
[-0.376  0.586]
[-0.391  0.53 ]
[ 1.865  1.668]
[ 2.032  1.466]
[ 2.199  0.783]
[ 2.04   0.881]
[ 2.052  1.153]
[ 2.496  0.995]
[ 1.775  0.786]
[ 1.921  0.999]
[ 2.614  1.196]
[ 2.125  1.268]
[ 2.378  0.937]
```

```
[ 1.707  0.903]
[ 1.888  1.407]
[ 2.602  1.146]
[ 1.755  1.032]
[ 1.906  1.512]
[ 1.684  1.193]
[ 2.036  1.014]
[ 1.784  1.124]
[ 2.28   1.393]
[ 2.211  1.174]
[ 2.027  1.684]
[ 2.476  0.902]
[ 2.464  0.976]
[ 2.359  1.47 ]
[ 0.995  0.529]
[ 1.016  0.286]
[ 1.189  0.351]
[ 1.241 -0.038]
[ 1.378  0.512]
[ 1.145  0.801]
[ 1.259  0.064]
[ 0.972  0.488]
[ 1.     0.651]
[ 1.335  0.646]
[ 1.382  0.332]
[ 1.186  0.656]
[ 0.907  0.184]
[ 1.203  0.623]
[ 1.508  0.454]
[ 1.449  0.412]
[ 1.155  0.612]
[ 0.954  0.295]
[ 1.485  0.14 ]
[ 1.33   0.134]
[ 1.602  0.521]
[ 0.78   0.332]
[ 1.468  0.453]
[ 1.015  0.556]
[ 0.951  0.36 ]
[-0.226 -0.227]
[ 0.172 -0.076]
[ 0.413 -0.303]
[ 0.466  0.112]
[ 0.351 -0.089]
[ 0.545  0.177]
[ 0.643 -0.059]
[-0.131  0.177]
[ 0.22  -0.44 ]
```

```
 [ 0.441 -0.342]
 [ 0.022 -0.005]
 [-0.081 -0.356]
 [ 0.07  -0.14 ]
 [-0.052 -0.011]
 [ 0.249 -0.229]
 [ 0.527  0.132]
 [ 0.379 -0.131]
 [ 0.283 -0.139]
 [ 0.348 -0.159]
 [-0.126  0.198]
 [ 0.448  0.061]
 [ 0.02  -0.21 ]
 [ 0.181 -0.14 ]
 [ 0.341 -0.584]
 [ 0.353  0.02 ]
 [ 1.717 -2.079]
 [ 1.283 -1.657]
 [ 1.31  -1.778]
 [ 1.061 -1.973]
 [ 1.35  -1.994]
 [ 1.544 -1.552]
 [ 1.283 -1.884]
 [ 1.518 -1.461]
 [ 1.381 -1.678]
 [ 1.493 -1.214]
 [ 1.187 -1.71 ]
 [ 1.595 -1.541]
 [ 1.707 -1.674]
 [ 1.248 -2.172]
 [ 1.73  -1.977]
 [ 1.249 -1.992]
 [ 1.215 -1.746]
 [ 1.172 -1.972]
 [ 1.641 -1.879]
 [ 1.718 -1.647]
 [ 1.186 -1.365]
 [ 1.409 -1.622]
 [ 1.395 -2.119]
 [ 1.573 -1.939]
 [ 1.658 -1.749]], [[-0.547  0.087  0.712]
 [-0.555 -0.675  0.499]
 [-0.587 -0.319  0.368]
 [-0.546 -0.457  0.924]
 [-0.551 -0.719  0.518]
 [-0.595 -0.322  0.8  ]
 [-1.305 -0.557  0.598]
 [-0.675 -0.221  1.144]
```

```
[-0.76  -0.269  0.933]
[-0.834 -0.592  0.658]
[-0.814  0.238  0.706]
[-0.552 -0.504  0.735]
[-0.968 -0.068  0.729]
[-0.507 -0.301  0.993]
[-0.882 -0.307  1.018]
[-0.733 -0.957  0.904]
[-0.926 -0.269  0.94 ]
[-1.077 -0.243  0.42 ]
[-0.934 -0.283  0.735]
[-0.776 -0.037  0.903]
[-0.44  -0.58   0.377]
[-0.584 -0.243  0.55 ]
[-0.32  -0.407  0.825]
[-0.803 -0.09   0.703]
[-0.794 -0.572  0.505]
[-1.633 -0.46   0.352]
[-1.303  0.187 -0.052]
[-1.058 -0.317  0.447]
[-0.984  0.269  0.447]
[-1.32  -0.223  0.537]
[-0.908  0.055  0.227]
[-1.557  0.062 -0.136]
[-1.174 -0.435  0.134]
[-1.227 -0.065 -0.06 ]
[-1.093 -0.735  0.147]
[-1.255 -0.057  0.505]
[-1.492  0.086  0.151]
[-1.143 -0.553  0.493]
[-0.831 -0.11   0.077]
[-1.056 -0.483  0.595]
[-1.416 -0.466  0.477]
[-1.586 -0.495  0.11 ]
[-0.893 -0.133  0.365]
[-0.973 -0.439  0.311]
[-0.913 -0.014  0.299]
[-0.771  0.128  0.535]
[-1.387 -0.349  0.002]
[-1.653 -0.326 -0.069]
[-1.234 -0.452  0.287]
[-1.033 -0.503  0.251]
[-2.457 -0.018 -0.119]
[-2.501  0.146  0.068]
[-2.825  0.23   0.015]
[-2.2    0.268 -0.082]
[-2.673  0.687 -0.415]
[-2.728  0.551  0.02 ]
```

```
[-2.188 -0.267 -0.113]
[-2.485  0.265 -0.334]
[-2.22  -0.     0.203]
[-2.054  0.572 -0.158]
[-2.368  0.153  0.276]
[-2.505  0.096 -0.132]
[-2.631  0.506  0.299]
[-2.492  0.32  -0.342]
[-2.254 -0.228  0.112]
[-2.617 -0.023  0.198]
[-2.453  0.063 -0.317]
[-2.345 -0.238  0.03 ]
[-2.37  -0.106  0.111]
[-2.581  0.48   0.118]
[-2.466  0.335 -0.191]
[-2.656  0.228  0.251]
[-2.597  0.312 -0.396]
[-2.379  0.58  -0.154]
[-2.971  0.232  0.441]
[-0.404  0.322 -0.61 ]
[-0.984  0.791 -0.937]
[-0.442  1.017 -0.843]
[-0.429  0.428 -0.784]
[-0.983  0.564 -1.16 ]
[-0.464  0.611 -0.783]
[-0.789  0.97  -1.253]
[-0.262  0.829 -1.587]
[-0.308  0.33  -1.092]
[-0.337  0.552 -1.044]
[-0.339  0.437 -1.011]
[-0.729  0.723 -1.008]
[-0.338  0.836 -0.767]
[-0.77   0.465 -0.975]
[-0.297  0.955 -0.939]
[-1.01   0.609 -0.681]
[-0.58   1.035 -1.167]
[-0.838  0.72  -1.233]
[-0.343  0.343 -1.234]
[-0.334  0.936 -0.882]
[-0.738  1.154 -1.168]
[-1.002  0.759 -1.22 ]
[-0.6    0.571 -1.257]
[-0.376  0.586 -1.147]
[-0.391  0.53  -1.069]
[ 1.865  1.668  0.39 ]
[ 2.032  1.466  0.542]
[ 2.199  0.783  0.332]
[ 2.04   0.881  0.614]
```

```
[ 2.052  1.153  0.374]
[ 2.496  0.995  0.572]
[ 1.775  0.786  0.913]
[ 1.921  0.999  0.508]
[ 2.614  1.196  0.833]
[ 2.125  1.268  0.217]
[ 2.378  0.937  0.684]
[ 1.707  0.903  0.504]
[ 1.888  1.407  0.217]
[ 2.602  1.146  0.895]
[ 1.755  1.032  0.448]
[ 1.906  1.512  0.91 ]
[ 1.684  1.193  0.554]
[ 2.036  1.014  0.247]
[ 1.784  1.124  0.538]
[ 2.28   1.393  0.227]
[ 2.211  1.174  0.987]
[ 2.027  1.684  1.058]
[ 2.476  0.902  0.347]
[ 2.464  0.976 -0.06 ]
[ 2.359  1.47   0.715]
[ 0.995  0.529 -0.489]
[ 1.016  0.286 -0.078]
[ 1.189  0.351 -0.011]
[ 1.241 -0.038 -0.939]
[ 1.378  0.512 -0.373]
[ 1.145  0.801 -0.552]
[ 1.259  0.064 -0.159]
[ 0.972  0.488 -0.266]
[ 1.     0.651 -0.476]
[ 1.335  0.646 -0.315]
[ 1.382  0.332 -0.513]
[ 1.186  0.656 -0.409]
[ 0.907  0.184 -0.567]
[ 1.203  0.623 -0.272]
[ 1.508  0.454 -0.708]
[ 1.449  0.412 -0.515]
[ 1.155  0.612 -0.271]
[ 0.954  0.295 -0.728]
[ 1.485  0.14  -0.763]
[ 1.33   0.134 -0.536]
[ 1.602  0.521 -0.215]
[ 0.78   0.332 -0.397]
[ 1.468  0.453 -0.596]
[ 1.015  0.556 -0.723]
[ 0.951  0.36  -0.691]
[-0.226 -0.227  0.486]
[ 0.172 -0.076  0.507]
```

```
 [ 0.413 -0.303  0.321]
 [ 0.466  0.112  0.375]
 [ 0.351 -0.089  0.7  ]
 [ 0.545  0.177 -0.091]
 [ 0.643 -0.059  0.197]
 [-0.131  0.177  0.249]
 [ 0.22  -0.44  -0.262]
 [ 0.441 -0.342  0.459]
 [ 0.022 -0.005  0.257]
 [-0.081 -0.356  0.361]
 [ 0.07  -0.14   0.513]
 [-0.052 -0.011  0.237]
 [ 0.249 -0.229  0.63 ]
 [ 0.527  0.132  0.77 ]
 [ 0.379 -0.131  0.515]
 [ 0.283 -0.139  0.011]
 [ 0.348 -0.159  0.354]
 [-0.126  0.198  0.316]
 [ 0.448  0.061  0.337]
 [ 0.02  -0.21   0.436]
 [ 0.181 -0.14   0.005]
 [ 0.341 -0.584  0.365]
 [ 0.353  0.02   0.319]
 [ 1.717 -2.079 -0.265]
 [ 1.283 -1.657 -0.221]
 [ 1.31  -1.778 -0.401]
 [ 1.061 -1.973 -0.712]
 [ 1.35  -1.994 -0.221]
 [ 1.544 -1.552 -0.237]
 [ 1.283 -1.884 -0.042]
 [ 1.518 -1.461 -0.57 ]
 [ 1.381 -1.678 -0.482]
 [ 1.493 -1.214  0.097]
 [ 1.187 -1.71  -0.48 ]
 [ 1.595 -1.541 -0.258]
 [ 1.707 -1.674 -0.379]
 [ 1.248 -2.172 -0.174]
 [ 1.73  -1.977 -0.603]
 [ 1.249 -1.992 -0.57 ]
 [ 1.215 -1.746  0.009]
 [ 1.172 -1.972 -0.002]
 [ 1.641 -1.879 -0.492]
 [ 1.718 -1.647 -0.445]
 [ 1.186 -1.365 -0.262]
 [ 1.409 -1.622 -0.506]
 [ 1.395 -2.119 -0.605]
 [ 1.573 -1.939 -0.123]
 [ 1.658 -1.749 -0.599]]
```

[9]:
```
print(f"{compute_pca(X, n_components=7)}")
```

```
n_components:7 : 0.9049017075553293)
[[-0.547  0.087  0.712 …  0.184  0.012  0.27 ]
 [-0.555 -0.675  0.499 …  0.547 -0.263  0.064]
 [-0.587 -0.319  0.368 …  0.712  0.276  0.075]
 …
 [ 1.395 -2.119 -0.605 …  0.072  0.078 -0.522]
 [ 1.573 -1.939 -0.123 … -0.234 -0.023 -0.164]
 [ 1.658 -1.749 -0.599 … -0.315  0.261 -0.072]]
```

[10]:
```
print(f"{compute_pca(X, n_components=11)}")
```

```
n_components:11 : 0.9616681690332414)
[[-0.547  0.087  0.712 … -0.351  0.118 -0.224]
 [-0.555 -0.675  0.499 … -0.093 -0.057  0.148]
 [-0.587 -0.319  0.368 … -0.024  0.265  0.248]
 …
 [ 1.395 -2.119 -0.605 …  0.11  -0.071 -0.307]
 [ 1.573 -1.939 -0.123 …  0.162 -0.037  0.001]
 [ 1.658 -1.749 -0.599 … -0.022 -0.207 -0.349]]
```

Antwoord: 1. Alles behalve rood Fantasy (3rd person) en groen Fantasy (1st person). 2. 2D = 69% en 3D = 77%. 3. Je hebt 7 n_components om 90% van de informatie te bewaren. 4. Je hebt 11 n_components nodig om 95% van de informatie te bewaren.

1.2 Deel II: t-Distributed Stochastic Neighbour Embedding (t-SNE)

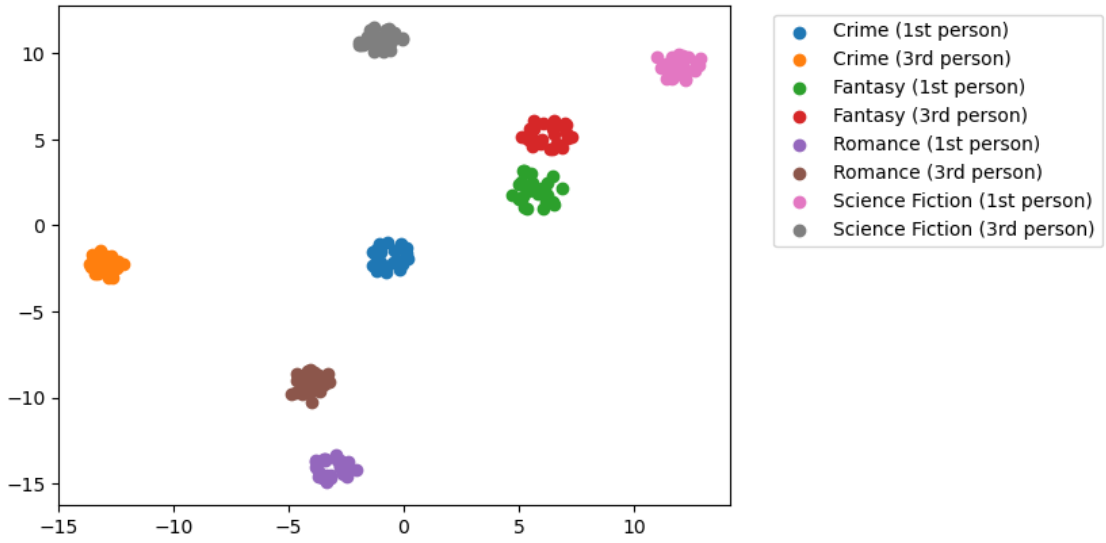
Een alternatieve methode voor dimensiereductie is *t-Distributed Stochastic Neighbour Embedding* (t-SNE).

1.2.1 Opdracht 4. Toepassing

Gebruik SciKit-Learn om met behulp van t-SNE de data tot 2 dimensies te reduceren en plot het resultaat (wederom met kleuren voor de categorien).

[11]:
```
# TODO: Schrijf hier je code.
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2)
X_tsne = tsne.fit_transform(X)
for label in np.unique(labels):
    mask = labels == label
    plt.scatter(X_tsne[mask, 0], X_tsne[mask, 1], label=label)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
```

[11]:
```
<matplotlib.legend.Legend at 0x1b77becc200>
```



1.2.2 Opdracht 5. Vergelijking

Vergelijk deze met de resultaten van je PCA-implementatie:

1. Hoe verhoudt de zichtbaarheid van de categorieën zich tussen beide resultaten?
2. Hoe verhouden de algoritmes zich in het behoud van informatie?

Beantwoord de volgende vragen los van de data van deze opdracht:

3. Waarvoor zou je PCA en t-SNE inzetten als je te maken krijgt met een onbekende (mogelijk ongelabelde) dataset?
4. Geef een voorbeeld waar PCA de voorkeur heeft boven t-SNE.
5. Geef een voorbeeld waar t-SNE de voorkeur heeft boven PCA.

Antwoord: 1. Met t-SNe kun je alle categoriën differentiëren. Bij PCA je dit niet doen voor alle categoriën. 2. Bij PCA weet je hoeveel informatie er verloren gaat en bij t-SNE wordt de lokale verhouding tussen de datapunten behouden.

3. Ik zou t-SNE gebruiken om features bij elkaar te clusteren en ik zou PCA gebruiken als ik wil weten hoeveel informatie behouden/verloren wordt.
4. Ik zou PCU gebruiken als ik met een dataset wil kijken naar feature-importance en probeer een zo goed mogelijk en efficiënt model te maken.
5. Ik zou t-SNE gebruiken als ik bepaalde woorden die vaak voorkomen in een bepaalde context wil clusteren.

1.2.3 Opdracht 6. Project

Als het goed is, heb je op dit moment een eerste idee van de data waar je in het project mee gaat werken. Geef antwoord op onderstaande vragen.

1. Wat is de dimensionaliteit waar je mee te maken hebt?
2. Beschrijf hoe dimensionaliteitsreductie-algoritmen je kunnen helpen de data te verkennen.

Antwoord: 1. De dimensionaliteit is erg hoog: $3840 \times 2160 \times 3 = 24.883.200$ dimensies voor één plaatje. 2. Ik zou beiden kunnen proberen. Ik zou kunnen beginnen met PCA en de belangrijkste features kunnen kiezen en ik zou met t-SNE clusters van bepaalde soorten schade kunnen ontdekken.

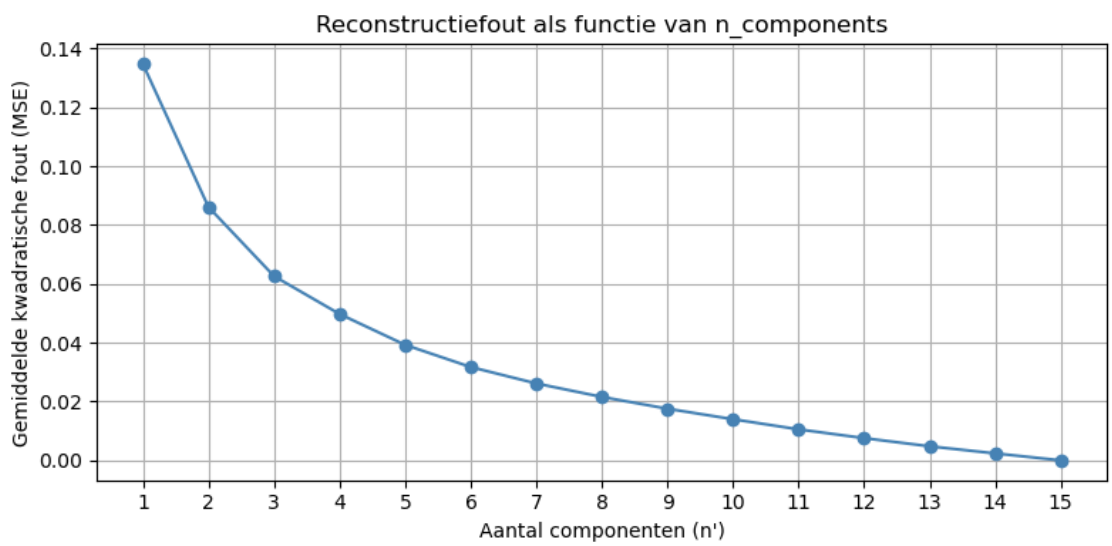
2 Opdracht voor Boven Niveau

Ik vroeg me af of er ook nog een mogelijkheid was om de dimensionaliteitsreductie om te keren. Dit bleek mogelijk te zijn: door de gereduceerde data stap voor stap terug te rekenen, kun je een benadering van de originele data terugkrijgen (bron: https://jakevdp.github.io/PythonDataScienceHandbook/05.09-principal-component-analysis.html). Hoe meer componenten je gebruikt bij de reductie, hoe kleiner de fout in de reconstructie — dit is te zien in onderstaande plot.

[ ]:
```
def compute_pca_reconstruct(X, n_components):
    arr = np.array(X, dtype=float)

    # Stap 1: Centreren
    mean = np.mean(arr, axis=0)
    arr_centered = arr - mean

    # Stap 2 & 3: Covariantiematrix + Eigenvectoren
    cov_matrix = np.cov(arr_centered, rowvar=False)
    eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

    # Stap 4: Selecteer de beste n_components eigenvectoren
    sorted_indices = np.argsort(eigenvalues)[::-1]
    selected = eigenvectors[:, sorted_indices[:n_components]]

    # Stap 5: Reduceer
    X_reduced = arr_centered @ selected

    # INVERSE STAP
    X_reconstructed = X_reduced @ selected.T + mean

    return X_reduced, X_reconstructed


# MSE berekenen
n_range = range(1, X.shape[1] + 1)
mse_values = []

for n in n_range:
    _, X_reconstructed = compute_pca_reconstruct(X, n_components=n)
    mse = np.mean((X - X_reconstructed) ** 2)
    mse_values.append(mse)
    print(f"n_components={n:2d}  →  MSE: {mse:.6f}")
```

```python
# Plot reconstructiefout
plt.figure(figsize=(8, 4))
plt.plot(list(n_range), mse_values, marker='o', color='steelblue')
plt.xlabel("Aantal componenten (n')")
plt.ylabel("Gemiddelde kwadratische fout (MSE)")
plt.title("Reconstructiefout als functie van n_components")
plt.xticks(list(n_range))
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
n_components= 1  →  MSE: 0.134759
n_components= 2  →  MSE: 0.086140
n_components= 3  →  MSE: 0.062632
n_components= 4  →  MSE: 0.049750
n_components= 5  →  MSE: 0.039266
n_components= 6  →  MSE: 0.031725
n_components= 7  →  MSE: 0.026144
n_components= 8  →  MSE: 0.021568
n_components= 9  →  MSE: 0.017558
n_components=10  →  MSE: 0.013982
n_components=11  →  MSE: 0.010538
n_components=12  →  MSE: 0.007555
n_components=13  →  MSE: 0.004781
n_components=14  →  MSE: 0.002366
n_components=15  →  MSE: 0.000000
```



Je kunt zien dat de fout het sterkst daalt in de eerste paar componenten, van 0.13 bij 1 component naar al 0.06 bij 3 componenten. Daarna vlakt de curve steeds meer af, wat betekent dat extra

componenten steeds minder nieuwe informatie toevoegen aan de reconstructie.